

Analysis of Data Leakage in Earthquake Magnitude Prediction: A Response Paper

Utku Efe Akdoğan, Aydın Gerek, Çağrı Diner*, Erdem Ata, Batuhan Kalem, Nurcan Meral Özel
Boğaziçi University
Email: cagri.diner@bogazici.edu.tr

Abstract—This paper presents a critical analysis of data leakage in a recent earthquake prediction study that reported 97.97% accuracy using machine learning models. The investigation reveals significant methodological concerns in temporal data handling. The primary issue stems from the implementation of random data splitting instead of chronological separation, which violates the temporal nature of earthquake data by allowing future information to leak into the training set. Through empirical reproduction of the study’s methodology, this work demonstrates how these data leakage issues artificially inflate performance metrics. When corrected for proper temporal validation, the best performing model used in the study (Random Forest) drops from 97.97% to 21.3% accuracy. This analysis underscores the critical importance of proper temporal data handling in geoscience applications and provides specific recommendations for preventing data leakage in future studies.

I. INTRODUCTION

Data leakage in machine learning refers to the use of information from outside the training dataset during model training or selection [3]. This issue is particularly critical in temporal data analysis, where future information must not influence the training process. In this paper, we analyze a recent study [1] that achieved 97.97% accuracy in earthquake magnitude prediction, examining potential data leakage issues that may affect the validity of these results. The full implementation and dataset can be found at the following repository: GitHub.

II. TEMPORAL DATA AND DATA LEAKAGE

A. Definitions

Temporal data requires special consideration in machine learning applications. According to [2], temporal leakage occurs when an ML model uses future information to predict past events, or when the training process incorporates information that would not be available at the time of prediction. Data leakage can manifest in three primary forms [4]:

- **Overlap Leakage:** When test data directly or indirectly influences the training process.
- **Multi-test Leakage:** When test data is used repeatedly for evaluation.
- **Preprocessing Leakage:** When test data influences the transformation of training data.

B. Correct Methodology for Temporal Splitting

For temporal data, the proper methodology requires [2]:

- **Strict chronological separation** between training and test sets.

- No future information leaking into the training process.
- Independent preprocessing of training and test sets.

III. DATA LEAKAGE ANALYSIS

A. Identified Issues

The analysis reveals critical concerns regarding data leakage in the study’s methodology:

1) Temporal Splitting Implementation:

- The investigation revealed the use of random splitting (`random_state=15`) in the model parameters, as reproducibility experiments with this random state yielded identical results to those reported in the original study.
- This random splitting approach fundamentally violates the temporal nature of earthquake data by allowing future information to leak into the training set.
- The study’s documentation lacks any discussion of temporal considerations in their validation strategy.

2) Model Selection and Implementation Concerns:

- The reported performance of Random Forest (97.97%) surpassing both XGBoost (96.8%) and LightGBM (93.4%) raises questions, as these gradient boosting methods typically demonstrate superior performance.
- The use of Transformer models with a relatively small dataset (approximately 23,000 samples) requires careful consideration, as these models typically require large datasets to train effectively and achieve robust generalization.

B. Impact on Results

The lack of proper temporal data splitting may have contributed to the unusually high accuracy (97.97%) reported for the Random Forest in the study [1]. When the same model was reproduced using identical hyperparameters but with proper temporal splitting, the accuracy drops dramatically to 21.3%. A similar decline in performance is observed across all models when proper temporal validation is applied. For example, the gradient boost models, XGBoost and LightGBM, experience substantial drops in accuracy, achieving 20.8% and 21.4%, respectively. Notably, this performance is even worse than majority class guessing (27.7%), suggesting that the original high accuracy was primarily due to data leakage rather than genuine predictive performance. A comprehensive summary of these evaluations is provided in Table 1 in the Appendix.

As noted by [3], preprocessing leakage alone can significantly inflate performance metrics, particularly when feature

selection is performed on the entire dataset before splitting. The consistent degradation across all model architectures further supports the hypothesis that the original high accuracies were primarily due to data leakage rather than meaningful pattern recognition in the seismic data.

IV. RECOMMENDATIONS

To address these issues, we recommend:

- Implementing strict temporal splitting with a clear cutoff date.
- Performing all preprocessing steps independently on training and test sets.
- Using time-series cross-validation techniques.
- Documenting all data handling procedures explicitly.

V. CONCLUSION

While the original study presents interesting approaches to earthquake prediction, our analysis suggests that data leakage may have influenced the reported results. Future work in this domain should carefully consider temporal aspects in data splitting and preprocessing to ensure reliable model evaluation.

APPENDIX

This section provides detailed insights into the dataset and methodology implemented as described in the referenced paper [1]. Below are the details of the dataset and methods:

A. Dataset Description

The study by Yavas et al. [1] utilized earthquake data from the Southern California Earthquake Data Center (SCEDC), focusing on the Los Angeles region:

- Geographic scope: Events within 100 km radius of Los Angeles (34.0522°N, 118.2437°W)
- Time period: January 1, 2012 to September 01, 2024
- Magnitude types: Four different types (Ml, Mh, Mw, Mlr)
- Event distribution: 23,284 Ml events, 52 Mh events, 52 Mw events, and 2 Mlr events

B. Feature Engineering Details

The study by Yavas et al. [1] implemented the following engineered features:

- 1) Rolling Mean of Depth (d_i):
 - Calculated for the last 30 days up to each event
 - Formula: $d_i = \frac{1}{N_i} \sum_j d_j$
 - Where N_i is the number of events in the last 30 days up to event i
- 2) Time Since Last Earthquake ($T_{\text{since last}}$):
 - Measures intervals between successive earthquakes
 - Provides insights into frequency and recurrence patterns
 - Calculated chronologically for each event
- 3) Gutenberg-Richter b-value:
 - Calculated based on previous 50 events
 - Uses the formula: $N(M) = 10^{a-bM}$
 - Where $N(M)$ is the number of earthquakes with magnitudes $\geq M$

4) Incremental b-values:

- $\Delta b_{i,i-2}$: b-value changes between events i and $i-2$
- $\Delta b_{i-2,i-4}$: b-value changes between events $i-2$ and $i-4$
- $\Delta b_{i-4,i-6}$: b-value changes between events $i-4$ and $i-6$
- $\Delta b_{i-6,i-8}$: b-value changes between events $i-6$ and $i-8$
- $\Delta b_{i-8,i-10}$: b-value changes between events $i-8$ and $i-10$

5) Maximum magnitude recorded during the last week ($M_{\text{max}}^{\text{lastweek}}$):

- Maximum magnitude among all events occurring within 7 days prior to each event
- Calculated as $M_{\text{max}}^{\text{lastweek}} = \max\{M_j \mid t_i - 7 \text{ days} \leq t_j < t_i\}$

6) Clustering coefficient (Cclust):

- Measure of the degree to which points in a dataset tend to cluster together
- Quantifies the spatial clustering of earthquake events

7) Standard Deviation of Magnitude ($\sigma_{\text{mag},30}$):

- Variability in earthquake magnitudes over 30 days
- Indicates stability/instability of seismic patterns

8) Mean Square Deviation (η):

- Sum of the mean square deviation from the regression line based on the Gutenberg-Richter law
- Calculated as $\eta = (1/n) \sum (d_i^2)$, where d_i is the deviation between observed and expected logarithmic earthquake counts
- Used to measure prediction reliability and model fit

9) Magnitude Difference (ΔM):

- Difference between the largest observed magnitude and largest expected magnitude based on the Gutenberg-Richter law

10) Elapsed time since the last earthquake (T):

- Total time interval between the first and last event within a specified window of n events
- Calculated as $T = t_n - t_1$, where t_1 is the time of first event and t_n is the time of n -th event

11) Mean time between events (μ):

- Average time interval between successive earthquakes
- Calculated as $\mu = T/n$, where T is total elapsed time and n is number of events

12) Coefficient of variation of the inter-event times (CV):

- Statistical measure of inter-event time variability
- Normalized measure of time pattern dispersion

13) Square root of cumulative seismic energy ($\sqrt{E\Sigma}$):

- Measure of total energy released by earthquakes
- Calculated as square root of sum of seismic energy from all events

14) Number of earthquakes in the last 30 days ($N_{eq,30}$):

- Count of seismic events within 30-day window
- Rolling count updated for each event

C. Target Variable Definition

The target variable for this study is the maximum earthquake magnitude predicted within the next 30 days [1]. The target variable was defined as follows:

- Target scope: Maximum magnitude of the next seismic event occurring within 30 days of each event.
- Classification method: Magnitudes were categorized into six classes using the Jenks Natural Breaks optimization method

D. Model Implementation

The study evaluated two categories of algorithms:

1) Traditional Machine Learning Models:

- Logistic Regression
- Decision Trees
- Random Forest
- Gradient Boosting (GBM, XGBoost, LightGBM)
- Support Vector Machines (SVM)
- k-Nearest Neighbors (k-NN)
- Naive Bayes
- AdaBoost

2) Neural Network Models:

- Multilayer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Long Short-Term Memory Networks (LSTM)
- Gated Recurrent Units (GRU)
- Transformer Models

E. Selected Features

The final model used 15 features, achieving 97.97% accuracy:

- 1) $N_{eq,30}$: Number of Earthquakes in the Last 30 Days
- 2) d_i : Rolling Mean of Depth
- 3) C_{clust} : Clustering Coefficient
- 4) dE_{12} : Change in energy release patterns
- 5) $\sigma_{mag,30}$: Standard Deviation of Magnitude
- 6) T : Elapsed time between the last n events
- 7) CV : Coefficient of variation of the inter-event times
- 8) b -value: Gutenberg-Richter b -value for the moment magnitude
- 9) ΔM : Difference between largest observed and expected magnitudes
- 10) η : Sum of the mean square deviation from regression line
- 11) $M_{max}^{last\ week}$: Maximum magnitude recorded during the last week
- 12) $\Delta b_{i-8,i-10}$: Incremental b -value between events $i-8$ and $i-10$
- 13) Longitude: Longitude of the earthquake epicenter
- 14) $\Delta b_{i-6,i-8}$: Incremental b -value between events $i-6$ and $i-8$
- 15) Latitude: Latitude of the earthquake epicenter

F. Training Strategy

- Data split: 60% training, 20% validation, 20% test
- Random state: 15
- Feature selection: 15 selected features as detailed in section E.

G. Final Random Forest Model Configuration

- bootstrap: False
- max_depth: 30
- min_samples_leaf: 1
- min_samples_split: 2
- n_estimators: 100

DATA AVAILABILITY

The dataset used in the original study analyzed in this response paper is publicly available at Zenodo and can be accessed via the provided link. The data, the code and implementation details for our analysis are available at GitHub repository.

SUPPLEMENTARY INFORMATION

No supplementary materials were used in this study.

REFERENCES

- [1] C. E. Yavas, L. Chen, C. Kadlec, and Y. Ji, "Improving earthquake prediction accuracy in Los Angeles with machine learning," *Scientific Reports*, vol. 14, no. 24440, pp. 1–54, 2024. DOI: <https://doi.org/10.1038/s41598-024-76483-x>.
- [2] S. Kapoor and A. Narayanan, "Leakage and the Reproducibility Crisis in ML-based Science," arXiv preprint arXiv:2207.07048, Jul. 2022. [Online]. Available: <https://arxiv.org/abs/2207.07048>
- [3] N. Bussola, A. Marcolini, V. Maggio, G. Jurman, and C. Furlanello, "AI slipping on tiles: data leakage in digital pathology," in *Pattern Recognition: ICPR International Workshops and Challenges*, A. Del Bimbo, S. Sclaroff, T. Mei, H. J. Escalante, R. Cucchiara, G. M. Farinella, M. Bertini, and R. Vezzani, Eds., Cham: Springer, vol. 12661, pp. 167–182, 2021, [Online]. Available: <https://doi.org/10.48550/arXiv.1909.06539>.
- [4] C. Yang, R. A. Brower-Sinning, G. A. Lewis, and C. Kästner, "Data Leakage in Notebooks: Static Detection and Better Processes," in *37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*, Rochester, MI, USA, October 10–14, 2022, ACM, New York, NY, USA, 12 pages. [Online]. Available: <https://doi.org/10.1145/3551349.3556918>.

AUTHOR CONTRIBUTIONS

Ç.D. and A.G. contributed to the conception and design of the work. Ç.D., as an expert in both geophysics and machine learning methods, played a key role in bridging the two fields. A.G. and E.A. provided an expert opinion on temporal data leakage and potential overfitting due to data leakage. B.K. and U.E.A. performed the analysis and applied machine learning methods, including fine-tuning hyperparameters. N.M.O. provided an expert opinion in the field of geophysics. All authors contributed to the interpretation of results, manuscript revisions, and approved the final submitted version.

COMPETING INTERESTS

The authors declare no competing financial or non-financial interests.

TABLE I
COMPARISON OF MODEL PERFORMANCE WITH AND WITHOUT DATA LEAKAGE, GROUPED BY RESULTS

Model	Metric	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	Reported	0.326	0.345	0.326	0.300	0.626
	Reported Results (reproduced)	0.326	0.345	0.326	0.300	0.626
	Corrected Results	0.222	0.278	0.222	0.175	0.559
Decision Trees	Reported	0.897	0.897	0.897	0.897	0.935
	Reported Results (reproduced)	0.897	0.897	0.897	0.897	0.935
	Corrected Results	0.208	0.200	0.208	0.193	0.513
Random Forest	Reported	0.977	0.977	0.977	0.977	0.999
	Reported Results (reproduced)	0.977	0.977	0.977	0.977	0.999
	Corrected Results	0.213	0.184	0.213	0.172	0.509
Gradient Boosting Machines	Reported	0.701	0.717	0.701	0.701	0.920
	Reported Results (reproduced)	0.701	0.716	0.701	0.701	0.920
	Corrected Results	0.199	0.195	0.199	0.179	0.489
Support Vector Machines	Reported	0.521	0.584	0.521	0.507	0.812
	Reported Results (reproduced)	0.521	0.584	0.521	0.507	0.812
	Corrected Results	0.190	0.189	0.190	0.143	0.475
k-Nearest Neighbors	Reported	0.638	0.648	0.638	0.637	0.870
	Reported Results (reproduced)	0.638	0.648	0.638	0.637	0.870
	Corrected Results	0.205	0.196	0.205	0.187	0.505
Naive Bayes	Reported	0.262	0.295	0.262	0.242	0.574
	Reported Results (reproduced)	0.262	0.295	0.262	0.242	0.574
	Corrected Results	0.229	0.236	0.229	0.197	0.542
AdaBoost	Reported	0.353	0.442	0.353	0.323	0.648
	Reported Results (reproduced)	0.353	0.442	0.353	0.323	0.648
	Corrected Results	0.311	0.247	0.311	0.246	0.593
XGBoost	Reported	0.968	0.968	0.968	0.967	0.998
	Reported Results (reproduced)	0.901	0.903	0.901	0.901	0.987
	Corrected Results	0.208	0.191	0.208	0.171	0.495
LightGBM	Reported	0.934	0.934	0.934	0.934	0.994
	Reported Results (reproduced)	0.934	0.934	0.934	0.934	0.994
	Corrected Results	0.214	0.196	0.214	0.185	0.500
Multilayer Perceptron	Reported	0.926	0.926	0.926	0.926	0.990
	Reported Results (reproduced)	0.923	0.924	0.923	0.923	0.990
	Corrected Results	0.200	0.182	0.200	0.182	0.497
Convolutional Neural Networks	Reported	0.626	0.656	0.626	0.645	0.896
	Reported Results (reproduced)	0.642	0.658	0.642	0.641	0.897
	Corrected Results	0.195	0.185	0.195	0.181	0.502
Recurrent Neural Networks	Reported	0.768	0.765	0.768	0.762	0.949
	Reported Results (reproduced)	0.750	0.759	0.750	0.748	0.947
	Corrected Results	0.182	0.165	0.182	0.164	0.480
Long Short-Term Memory	Reported	0.453	0.478	0.453	0.444	0.753
	Reported Results (reproduced)	0.476	0.495	0.476	0.468	0.774
	Corrected Results	0.183	0.180	0.183	0.170	0.495
Gated Recurrent Units	Reported	0.536	0.544	0.536	0.534	0.828
	Reported Results (reproduced)	0.551	0.559	0.551	0.550	0.835
	Corrected Results	0.193	0.205	0.193	0.179	0.490
Transformer Models	Reported	0.460	0.489	0.460	0.453	0.762
	Reported Results (reproduced)	0.469	0.483	0.469	0.458	0.768
	Corrected Results	0.194	0.190	0.194	0.170	0.483